

Software Design Description 102

ESP32-CAM – Capture Pipeline

Overview

The capture pipeline is the real-time core of the ESP32-CAM. It runs from wake through stop command, pulling frames from the OV2640 sensor, pushing each one downstream over SPI DMA, and copying every 10th frame into the PSRAM save ring buffer. The entire pipeline operates without touching the SD card or encoding any JPEG — those happen later in the post-capture phase (see SDD101).

Two FreeRTOS tasks cooperate: a **capture task** that pulls frames from the camera and enqueues them, and a **transmit task** that drains the queue over SPI. A PSRAM buffer pool provides zero-copy handoff between them.

OV2640 Sensor Configuration

Hardware Config

Parameter	Value
Pixel format	PIXFORMAT_GRAYSCALE
Resolution	320x240 (QVGA)
XCLK	20 MHz
Framebuffers	3 (PSRAM)
Grab mode	CAMERA_GRAB_LATEST (new design)
Interface	DVP parallel (8-bit data bus, VSYNC, HREF, PCLK)

Grab mode note: The current code uses `CAMERA_GRAB_WHEN_EMPTY`, which returns the oldest buffered frame. The new design switches to `CAMERA_GRAB_LATEST` with `fb_count=3`, which discards stale frames and always returns the most recent capture. This is critical for the diff-based compression pipeline — stale frames increase inter-frame delta energy and hurt compression ratio.

Locked Exposure

All auto controls are disabled at boot. The compression pipeline requires stable exposure across the burst so that inter-frame differences reflect real motion, not AEC/AGC drift.

```
#define OV2640_LOCKED_DEFAULTS {  
    .aec_enable    = 0,    // manual exposure  
    .agc_enable    = 0,    // manual gain  
    .awb_enable    = 0,    // manual white balance  
    .aec2_enable   = 0,    // DSP AEC off  
    .gain          = 0,    // gain floored  
    .gain_ceiling  = GAINCEILING_2X,  
    .exposure      = 50,    // initial seed (overwritten by auto-tune)  
    .target_mean   = 110,   // auto-tune target brightness  
    .tune_tolerance = 5,    // converge when |mean - target| < 5  
}
```

Proportional AEC Auto-Tune

On wake, before capture begins, the sensor exposure is tuned to the current scene brightness using a proportional controller:

```
aec_next = aec_current * target / mean
```

1. Set initial AEC value (default 50)
2. Capture a frame, compute mean pixel brightness over all 76,800 pixels
3. Adjust AEC proportionally, clamp to [1, 1200]
4. Repeat until $|\text{mean} - \text{target}| < \text{tolerance}$
5. Converges in 3-5 iterations (~300-500 ms at 100 ms per iteration)

After convergence, exposure is locked for the entire burst. No further auto-adjustment occurs.

Known limitation: In full direct sunlight, AEC=1 still overexposes. Mechanical shade visor required for outdoor deployment.

Capture Task

The capture task runs on core 0 at priority 6, pulling frames from the OV2640 driver and feeding them into the SPI transmit queue and save ring buffer.

Frame Pacing

```
TickType_t last_wake = xTaskGetTickCount();

for (;;) {
    camera_fb_t *fb = esp_camera_fb_get();
    // ... enqueue to SPI, copy to save ring ...
    esp_camera_fb_return(fb);
    vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(FRAME_PERIOD_MS));
}
```

`vTaskDelayUntil` sleeps for the remainder of each 33 ms period (30 fps target). If a cycle overruns — the capture + memcpy + enqueue exceeds 33 ms — the next iteration runs immediately with no sleep. This gives soft real-time pacing without dropping frames on occasional long cycles.

Capture Loop Flow

```
esp_camera_fb_get()
    |
    v
+-- enqueue_frame() --> SPI TX queue --> spi_transmit_task
    |
+-- frame_count % SAVE_INTERVAL == 0?
    |   yes --> memcpy to save_ring (PSRAM)
    |   no  --> skip
    |
    v
esp_camera_fb_return()
    |
    v
vTaskDelayUntil (pace to 33 ms)
```

Frame Lifecycle

1. `esp_camera_fb_get()` — returns pointer to one of 3 PSRAM framebuffers filled by the OV2640 DMA
2. Frame data is copied into a buffer from the image pool (`image_buffer_alloc`)
3. The `image_data_t` struct (pointer + size) is pushed to the SPI queue
4. If this is a save frame, raw pixels are also `memcpy`'d to the save ring
5. `esp_camera_fb_return()` — releases the framebuffer back to the camera driver
6. The SPI transmit task later frees the image pool buffer after transmission

The camera framebuffer is held for only the duration of the two `memcpy` operations (~0.2 ms total). With 3 framebuffers, the OV2640 DMA always has at least 2 free targets while one is being read.

SPI Transmit Task

A dedicated FreeRTOS task drains the image queue and pushes frames over SPI DMA. It runs independently of the capture task, decoupling sensor timing from bus timing.

Task Structure

```
void spi_transmit_task(void *pvParameters) {
    QueueHandle_t image_queue = (QueueHandle_t)pvParameters;
    image_data_t img_data;

    for (;;) {
        xQueueReceive(image_queue, &img_data, portMAX_DELAY);
        spi_dma_send_image(img_data.buffer, img_data.size);
        image_buffer_free(img_data.buffer);
    }
}
```

- Blocks on `xQueueReceive` when idle — zero CPU when no frames pending
- After each SPI send, frees the image pool buffer (zero-copy: no intermediate allocation)
- Queue depth of 1: if the transmit task falls behind, the capture task drops the frame at `xQueueSend` (non-blocking with timeout 0)

Frame Protocol

Each frame is sent as a 12-byte header followed by raw pixel data in 4092-byte DMA chunks:

Field	Size	Value
Magic	4 bytes	0xCAFEDEED
Size	4 bytes	Frame size in bytes (76,800)
Checksum	4 bytes	XOR of all image bytes

Payload: 76,800 bytes in $\text{ceil}(76800/4092) = 19$ DMA chunks.

Bus Mutex

`spi_dma_send_image()` holds a FreeRTOS mutex for the full duration of header + payload transfer. This serves two purposes:

1. **Serialization** — only one frame on the bus at a time
2. **Safe teardown** — `spi_dma_deinit()` blocks on the same mutex, guaranteeing no in-flight DMA when the bus is released for post-capture SD access

```
xSemaphoreTake(ctx.bus_mutex, pdMS_TO_TICKS(500));
// ... header + 19 chunks ...
xSemaphoreGive(ctx.bus_mutex);
```

Mutex timeout is 500 ms. If the bus is stuck, the frame is dropped and logged.

PSRAM Buffer Pool

The image buffer pool provides pre-allocated PSRAM buffers for zero-copy frame handoff between the capture and transmit tasks.

Configuration

Parameter	Value
Buffer count	3
Buffer size	76,800 bytes (320x240 grayscale)
Total PSRAM	225 KB
Allocation	heap_caps_malloc(MALLOC_CAP_SPIRAM) at boot

Thread Safety

All alloc/free operations are guarded by a FreeRTOS mutex. The pool tracks:

- **in_use** bitmap — which buffers are currently allocated
- **peak_usage** — high-water mark for diagnostics

Alloc/Free Pattern

```
capture_task:  buf = image_buffer_alloc()
               memcpy(buf, fb->buf, fb->len)
               xQueueSend(queue, {buf, len})
               |
               v
transmit_task: xQueueReceive(queue, {buf, len})
               spi_dma_send_image(buf, len)
               image_buffer_free(buf)
```

No malloc/free at runtime. All buffers are pre-allocated at boot and recycled through the pool.

PSRAM Budget

Pool	Count	Size Each	Total	Purpose
Camera framebuffers	3	76.8 KB	230 KB	OV2640 DMA targets
Image buffer pool	3	76.8 KB	225 KB	SPI TX queue handoff
Save ring buffer	30	76.8 KB	2,304 KB	Deferred SD save (SDD101)
Total PSRAM available	36		~2.7 MB 8 MB	~34% utilized

Timing Analysis

Per-Frame Budget (33 ms at 30 fps)

Operation	Time	Notes
esp_camera_fb_get	~0 ms	Returns immediately (GRAB_LATEST)
memcpy to image pool	~0.1 ms	76.8 KB PSRAM-to-PSRAM
xQueueSend	~0 ms	Non-blocking
memcpy to save ring	~0.1 ms	Every 10th frame only
esp_camera_fb_return	~0 ms	

Operation	Time	Notes
Capture task total	~0.2 ms	Well within 33 ms budget

SPI Transfer (concurrent with capture)

Metric	Value
Frame + header	76,812 bytes
SPI clock	18 MHz
Transfer time	~34 ms
Frame period	33 ms
Utilization	~103% — SPI is the bottleneck

At 18 MHz, the SPI bus is slightly over-subscribed at 30 fps. Actual sustained rate will settle at 25-28 fps depending on DMA overhead and FreeRTOS scheduling. The `vTaskDelayUntil` pacing gracefully handles this — when SPI backs up, the queue-full drop path discards frames rather than stalling the capture task.

Burst Test Mode

When `TEST_BURST_CAPTURE` is defined, the pipeline bypasses SPI entirely and writes raw frames directly to SD:

Parameter	Value
Frames	30
Discard	First 5 (AEC settle)
Resolution	320x240 QVGA
Format	Raw grayscale (<code>.raw</code>)
Output	<code>/sdcard/img_NNNN.raw</code>

This mode is used to harvest real-world frame data for the simulation pipeline (`sim/oct/wht_fpga.m`). SD stays mounted for the entire capture — no SPI, no pin conflict.

Pending Changes

Change	Description
<code>fb_count</code> 2 to 3	Current code uses 2 framebuffers; new design needs 3
<code>GRAB_WHEN_EMPTY</code> to <code>GRAB_LATEST</code>	Prevent stale frames from inflating WHT deltas
<code>FRAMESIZE_VGA</code> to <code>FRAMESIZE_QVGA</code>	Production mode should match burst mode (320x240)
SPI clock 10 to 18 MHz	<code>SPI_CLOCK_SPEED</code> in <code>DMA_SPI_master.h</code>
Save ring buffer	New PSRAM ring in <code>image_buffer_pool.c</code> (see SDD101)
MISO command receive	Check <code>rx_buffer</code> after each SPI transaction (see SDD120)
Stop command handler	Break capture loop on STOP, enter post-capture phase

Source Files

File	Purpose	Status
src/camera_control.c	Capture loop, enqueue, frame pacing	Needs refactor (save ring, GRAB_LATEST, stop handler)
src/ov2640_config.c	AEC auto-tune, sensor config	Done
include/ov2640_config.h	Config struct, LOCKED_DEFAULTS macro	Done
src/DMA_SPI_master.c	SPI DMA master, frame protocol, bus mutex	Needs MISO rx, 18 MHz
include/DMA_SPI_master.h	Pin defines, clock config	Needs clock bump
src/image_buffer_pool.c	PSRAM buffer pool alloc/free	Needs save ring extension
include/image_buffer_pool.h	Pool config (count, size)	Done

References

- SDD100 – ESP32-CAM Overview
- SDD101 – SD Save Pipeline
- SDD120 – CAM-DevKit SPI Interface
- ESP-IDF Camera Driver